SCUDERIA TOR VERGATA

AI Race Engineer Copilot

Un assistente per l’ingegnere di pista, costruito su OCI Enterprise AI

Team Struffoli
Valerio Bernardi · Alfredo Grande
Samuele De Santis · Luca Gugliotta

ORACLE



HACKATHON CON ORACLE · 22/05/2026

Obiettivo del progetto

Un chatbot operativo a supporto della Scuderia Tor Vergata.

Contesto real-time

Analisi immediata dei dati di gara e del regolamento.

Nessuna allucinazione

Esposizione esclusiva di dati e numeri effettivamente recuperati.

Incertezza dichiarata

Assenza di informazioni notificata esplicitamente all'utente.



Scuderia Tor Vergata · box e pit lane

Due fonti, due formati

01 DATI DI GARA

Telemetria

Dati dinamici sulle condizioni del veicolo a ogni pit di gara.

JSON

02 DOCUMENTI

Regolamento

Dossier tecnico e normativo per le verifiche dei veicoli in gara.

PDF

Task progettuali

L’architettura del copilota sviluppata in tre task.

01 TASK 1

Server MCP

Sviluppo di tool di estrazione per analizzare i dati JSON della telemetria.

02 TASK 2

RAG normativo

Chunking dei PDF e verifica della conformità tramite ricerca semantica.

03 TASK 3

App unificata

Collegamento del chatbot con server e RAG, e web app Streamlit per porre query all’assistente.

Un LLM da solo non basta

Perché non mettere tutto nel prompt?

Abbiamo **due fonti di informazione molto diverse**: metterle tutte nel prompt renderebbe il sistema poco efficiente e poco controllabile.

TELEMETRIA **DOCUMENTI**

Token sprecati
Context window piena di dati inutili

Poco controllo
Il modello cerca da solo nel contesto

Valori inventati
Calcoli approssimativi e regole non verificate



Costruire un harness attorno all'LLM

Il modello non è più un chatbot.

Orchestratore

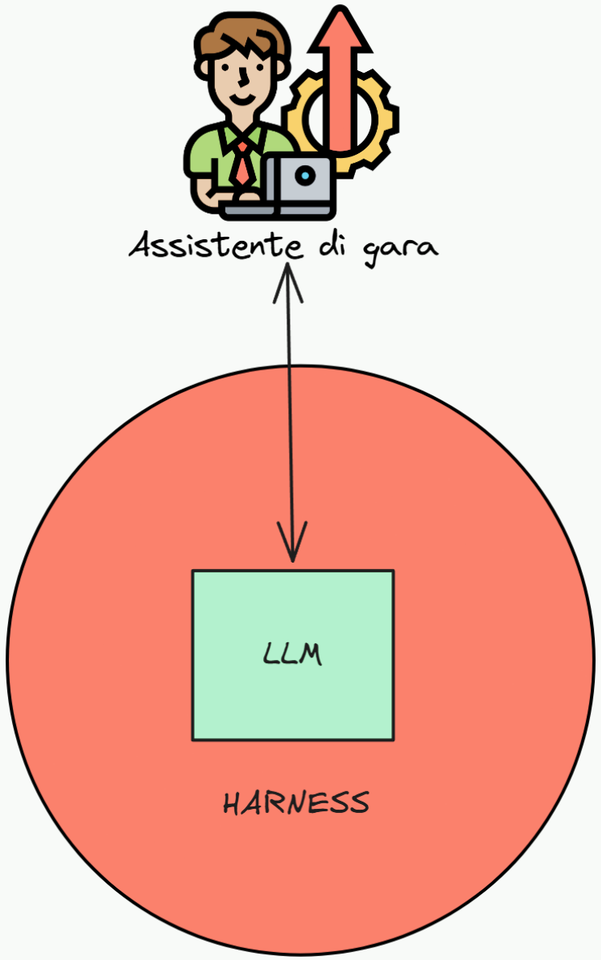
L'LLM coordina gli strumenti e ragiona

Tool dedicati

MCP + File Search

Separazione dei compiti

LLM e tool con responsabilità distinte



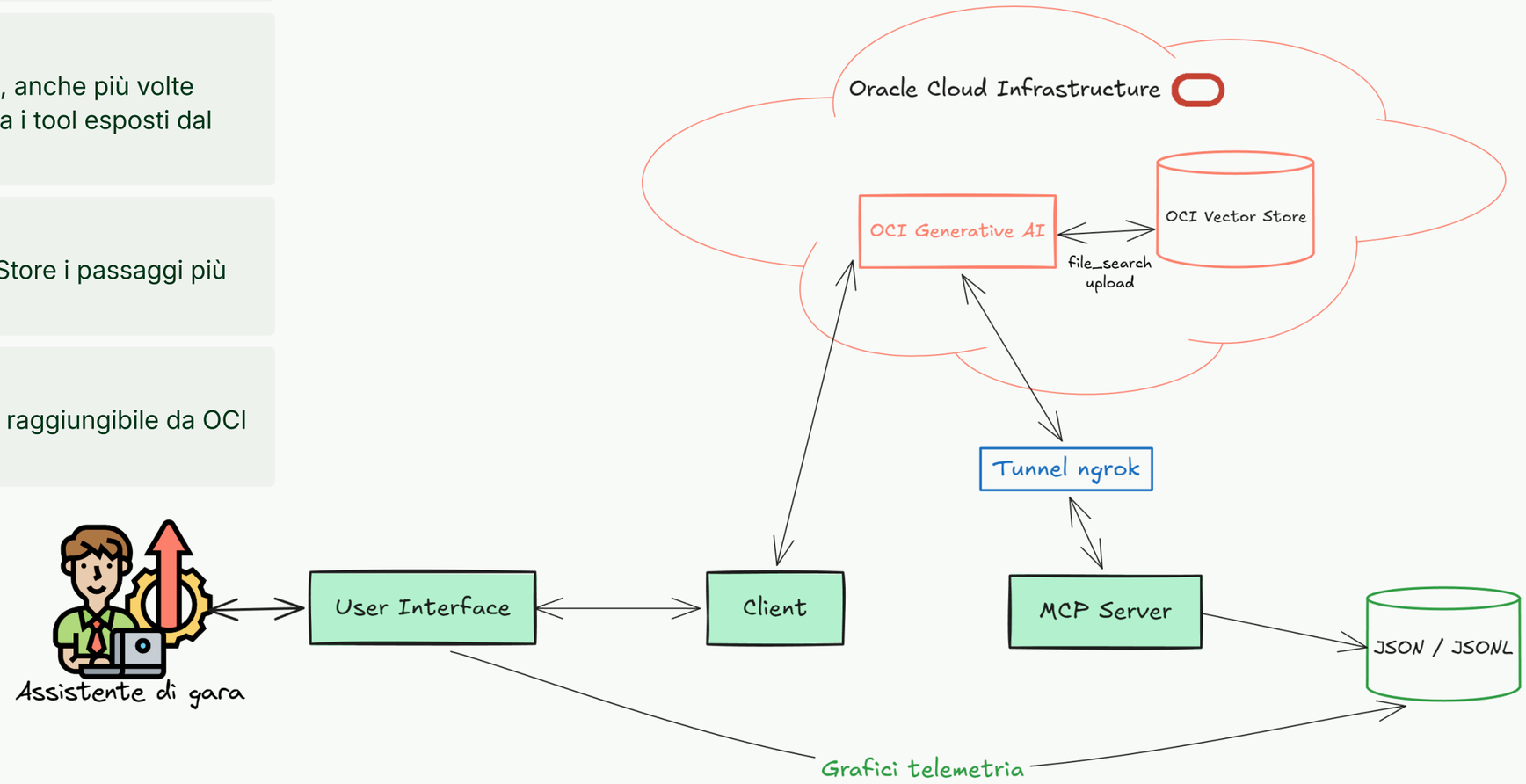
Una richiesta, due percorsi

OCI Generative AI
Riceve dal client, via Responses API, il prompt dell'utente e gli strumenti disponibili.

Tool calling
Il modello decide quali strumenti utilizzare, anche più volte nello stesso flusso. Per la telemetria invoca i tool esposti dal server MCP.

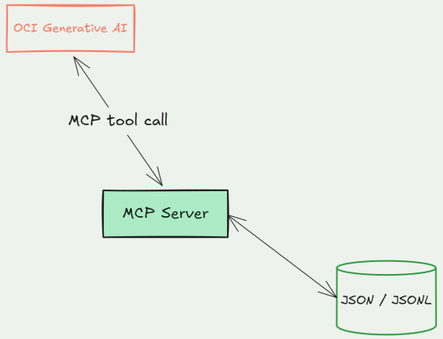
`file_search`
Tool di retrieval: recupera dall'OCI Vector Store i passaggi più rilevanti per la richiesta.

ngrok
Tunnel HTTPS: rende il server MCP locale raggiungibile da OCI Generative AI nel cloud.



I due strumenti: MCP e OCI Vector Store

01 DATI DI GARA

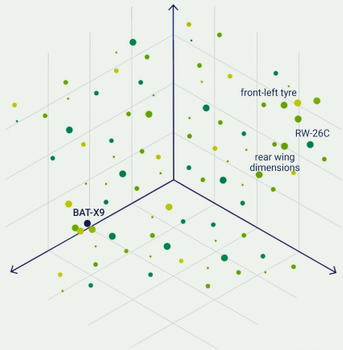


MCP Server

Espone tool per accedere in modo **controllato** a dati esterni ed eseguire calcoli deterministici.

- Telemetria in **JSON / JSONL**
- Standard per descrivere e invocare tool
- Chiamato da **OCI Generative AI** tramite MCP
- Calcoli deterministici eseguiti via codice

02 DOCUMENTI



OCI Vector Store

Embedding e ricerca semantica sui documenti.

- Ogni chunk diventa un **vettore multidimensionale**
- Significati simili → vettori **vicini nello spazio**
- **Ricerca semantica** tramite `file_search`
- Indicizzazione e retrieval gestiti da OCI

I tool MCP

mcp-server-struffoli.py TASK 1

```python
@mcp.tool()
def get_lap_window(start_lap: int,
                   end_lap: int) -> Dict[str, Any]:
    """Return a list of lap records between
    start_lap and end_lap, inclusive."""
    meta = _load_json("race_session_telemetry.json")
    result = []
    for lap in meta["laps"]:
        if start_lap <= lap["lap"] <= end_lap:
            result.append(lap)
    return {
        "start_lap": start_lap,
        "end_lap": end_lap,
        "laps_found": len(result),
        "laps": result
    }
```

— **Implementazione dell'MCP tool richiesto dal Task 1:** `get_lap_window`

mcp-server-struffoli.py TASK 3

```python
@mcp.tool()
def recommend_pit_strategy(
        current_lap: int) -> Dict[str, Any]:
    """Analizza strategia e usura per raccomandare
    il momento ottimale del pit stop e stimarne
    l'impatto sulla gara."""
    # calcolo deterministico
    laps_to_limit = (95.0 - current_wear) / deg_rate
    ...
    return {"recommendation": "PIT_IN_WINDOW",
            "target_pit_lap": 27, ...}
```

— **Implementazione di un MCP tool aggiunto per l'interfaccia (Task 3):** `recommend_pit_strategy`

L'OCI Vector Store

upload_file.py — PYTHON

```python
chunks = chunk_pdf_per_ai(pdf_path,
                          chunk_size=1500, overlap=200)
for i, chunk in enumerate(chunks):
    f = client.files.create(file=to_file(chunk),
                            purpose="user_data")
    client.vector_stores.files.create(
        vector_store_id=VECTOR_STORE_ID, file_id=f.id,
        attributes={"original_file": name,
                    "chunk_index": i})
```

— **Caricamento dei file**, divisi in chunk, nel Vector Store.

I file vengono divisi in **chunk di dimensione fissa**, per evitare di superare i limiti di memoria per file imposti dal Vector Store.

1500
caratteri per chunk

200
caratteri di overlap

Prompt Engineering

system prompt PROMPT

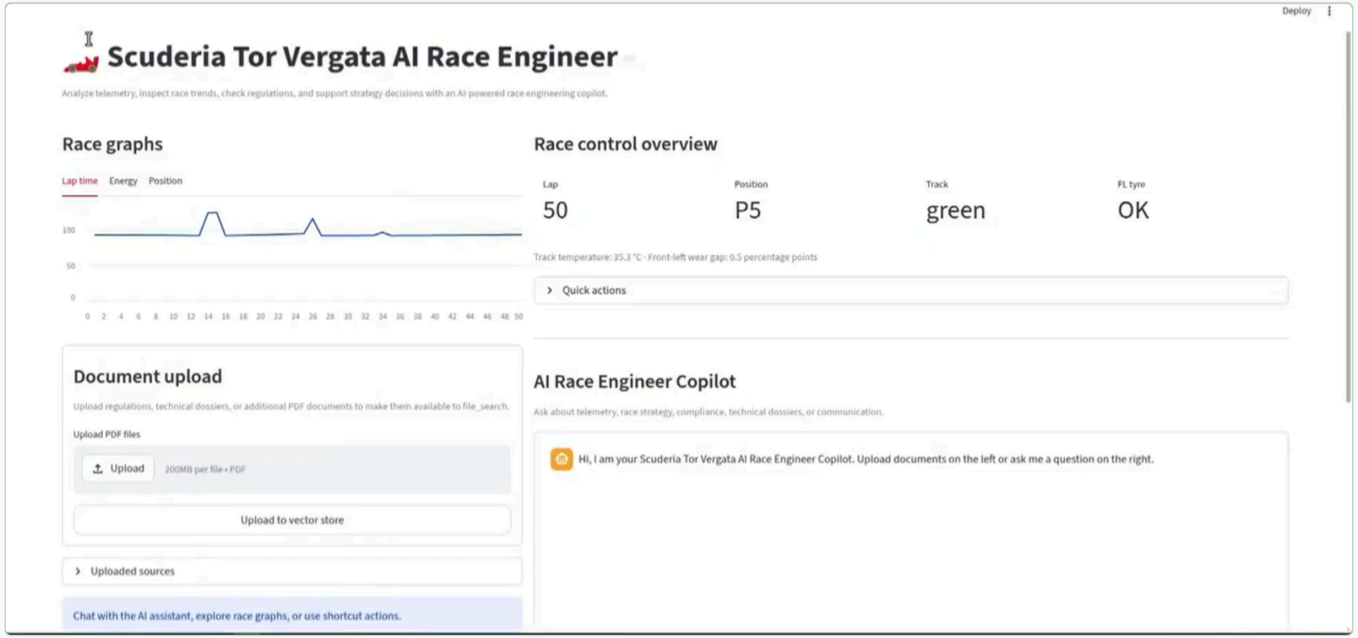
```
You are an AI Race Engineer Copilot for Scuderia Tor Vergata.
You combine two main capabilities:
1. Race telemetry and strategy analysis through MCP tools.
2. Regulatory compliance and dossier analysis via file_search.
...
Do not guess.
Use MCP tools for structured race data.
Use file_search for regulations, rulebooks, technical dossiers.
If evidence is missing, incomplete or ambiguous, say so explicitly.
Never invent:
- telemetry values
- lap data
...
For every answer, explicitly state which evidence source was used:
- MCP only
- file_search only
- MCP + file_search
- No tool evidence available
...
For compliance answers, provide a documentation support score:
how strongly the available documents support your answer,
NOT how legally compliant the component is.
```

REGOLE DEL COPILOTA

- Non inventare dati quando non sei sicuro della risposta
- Affidati ai tool MCP e `file_search`
- Dichiara la percentuale di informazioni presa da ogni fonte
- Mostra il ragionamento step by step che farebbe un ingegnere di gara

L’interfaccia del Race Engineer

Una dashboard Streamlit per consultare i dati di gara, interrogare l’assistente e caricare documenti tecnici.



interfaccia.py PYTHON

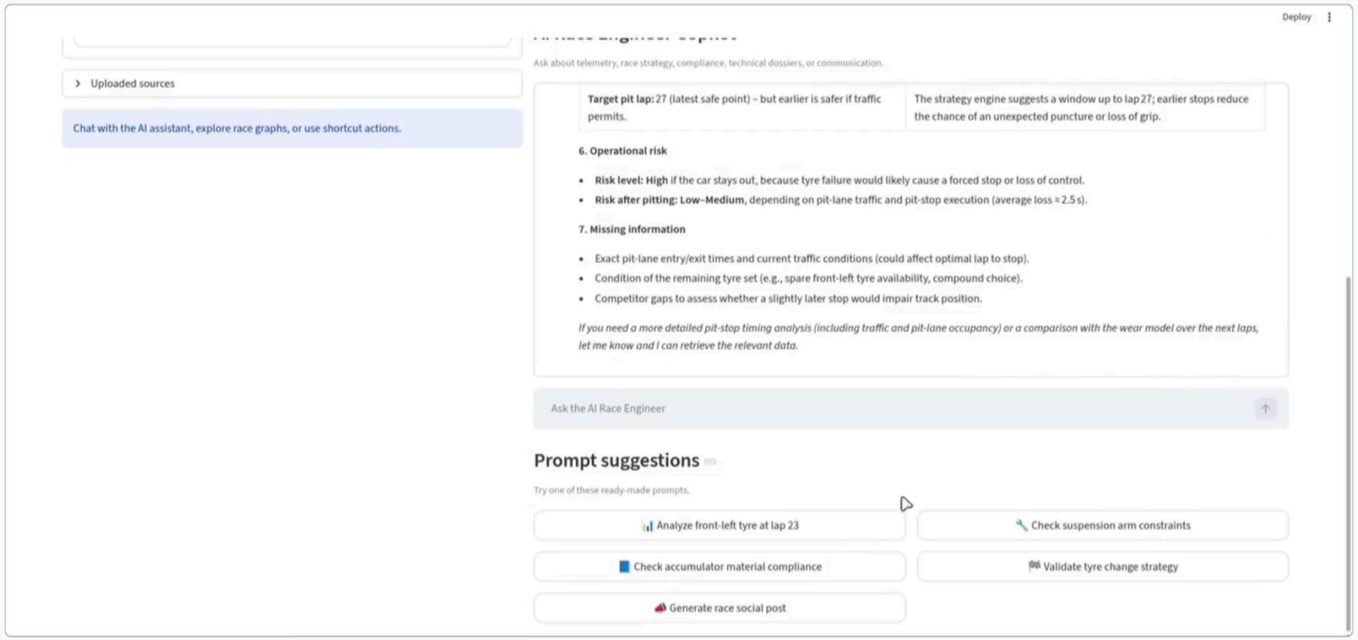
```python
tab1, tab2, tab3 = st.tabs(
    ["Lap time", "Energy", "Position"])

with tab1:
    if ("lap_time_s" in df.columns
            and
df["lap_time_s"].notna().any()):
        chart_df = (df.set_index("lap")
                    [["lap_time_s"]])
        st.line_chart(chart_df, height=180)
```

- **Un tab per grafico:** tempi, energia, posizione.

Dalla quick action alla risposta

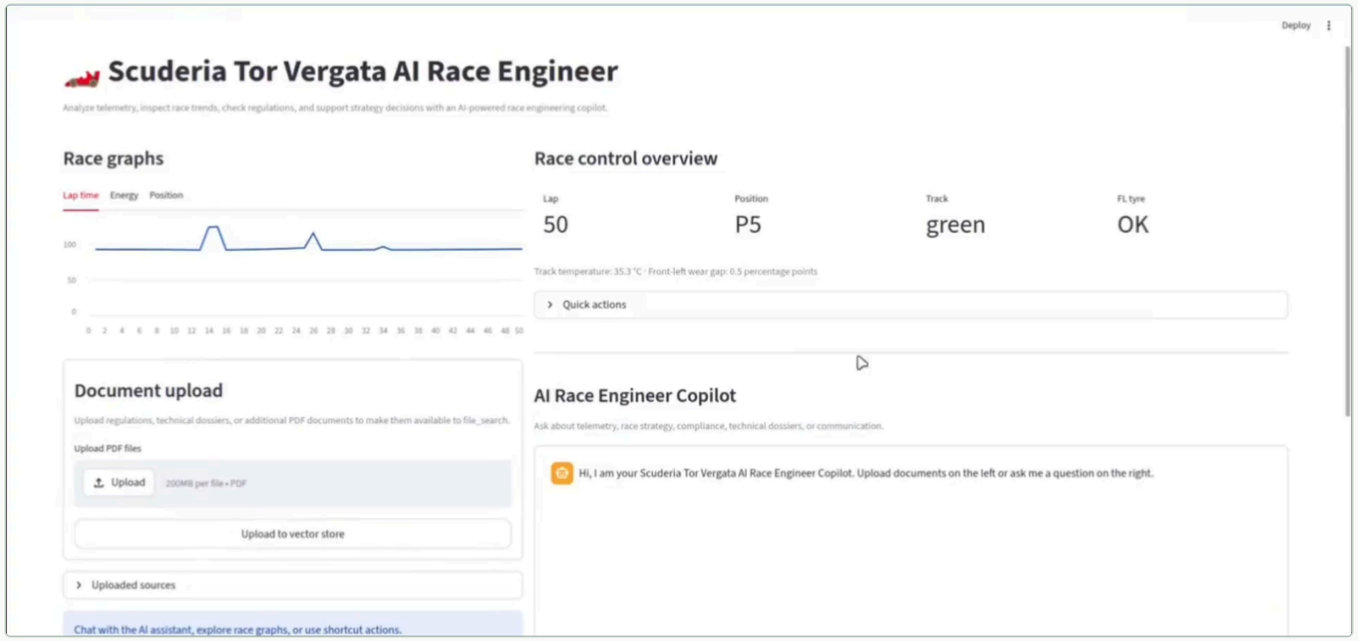
I dati della gara vengono recuperati tramite MCP e trasformati in una risposta operativa.



ANDATA	RITORNO
Streamlit	risultati MCP
↓	↓
client Python	OCI Responses API
↓	↓
OCI Responses API	client Python
↓	↓
strumenti MCP	chat Streamlit

Verifica documentale BAT-X9

Un secondo percorso per il modello: file_search apre l’accesso ai documenti.



ANDATA	RITORNO
Streamlit	passaggi recuperati
↓	↓
client Python	OCI Responses API
↓	↓
OCI Responses API	client Python
↓	↓
`file_search` → documenti	chat Streamlit

La fonte dipende dalla domanda

Una sola Responses API, due percorsi di evidenza.

01 TELEMETRIA

Analisi dello pneumatico e pit stop

100% MCP 0% file_search

DATO

Usura **61,4 %** contro **43,1 %** atteso

RISPOSTA

Sosta proposta tra i giri **24 e 27**

02 REGOLAMENTO

Verifica BAT-X9

100% file_search 0% MCP

DOCUMENTO TROVATO

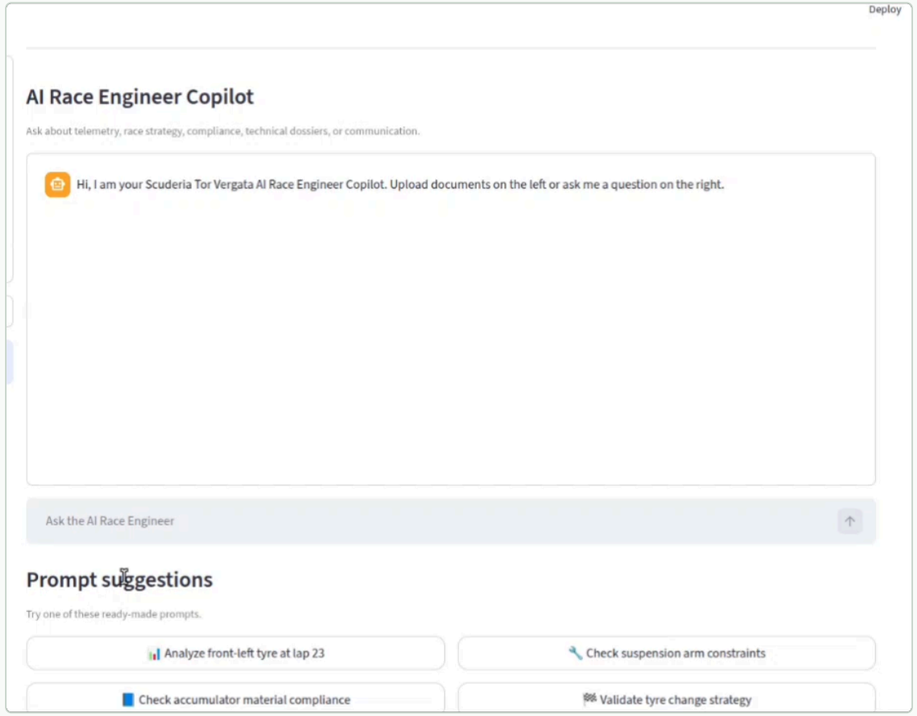
Dossier BAT-X9

RISPOSTA

Conformità non confermata: norma decisiva non recuperata

Dalla telemetria al post social

I dati in gara alimentano anche la comunicazione.



`get_session_overview` → ultimo giro

`get_lap_window` → ultimi 5 giri

Dati di gara → bozza del post

Grazie mille dell'attenzione

Team Struffoli

Valerio Bernardi · Alfredo Grande
Samuele De Santis · Luca Gugliotta
Scuderia Tor Vergata



Team Struffoli · Scuderia Tor Vergata